



# Gestion de Truffière

Application complète de gestion de truffière avec Docker, PostgreSQL, Node.js/Express et React.



## Prérequis

- Docker et Docker Compose installés
- Git (optionnel)



## Installation

### 1. Structure du projet

Créez la structure de dossiers suivante :

```
truffiere/  
├── docker-compose.yml  
├── init-db.sql  
├── backend/  
│   ├── Dockerfile  
│   ├── package.json  
│   └── server.js  
├── frontend/  
│   ├── Dockerfile  
│   ├── package.json  
│   ├── public/  
│   │   └── index.html  
│   └── src/  
│       ├── App.js  
│       ├── App.css  
│       ├── index.js  
│       └── components/  
│           ├── Dashboard.js  
│           ├── Parcelles.js  
│           ├── Arbres.js  
│           ├── Interventions.js  
│           ├── Recoltes.js  
│           ├── Clients.js  
│           ├── Ventes.js  
│           └── Statistiques.js  
└── nginx/  
    └── nginx.conf (optionnel pour production)
```

### 2. Copier les fichiers

Copiez tous les fichiers que je vous ai fournis dans les bons dossiers selon la structure ci-dessus.

### 3. Créer les fichiers manquants du frontend

#### frontend/public/index.html

```
html

<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Gestion de Truffière</title>
    <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
  </head>
  <body>
    <noscript>Vous devez activer JavaScript pour utiliser cette application.</noscript>
    <div id="root"></div>
  </body>
</html>
```

#### frontend/src/index.js

```
javascript

import React from 'react';
import ReactDOM from 'react-dom/client';
import './App.css';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

### 4. Lancer l'application

Dans le répertoire racine (`truffiere/`), exécutez :

```
bash

# Construire et démarrer tous les services
docker-compose up --build

# Ou en mode détaché (arrière-plan)
docker-compose up -d --build
```

### 5. Accéder à l'application

- **Frontend React** : <http://localhost:3000>
- **Backend API** : <http://localhost:3001/api>
- **PostgreSQL** : localhost:5432

## 6. Tester l'API

bash

*# Vérifier que l'API fonctionne*

`curl http://localhost:3001/api/health`

*# Récupérer les parcelles*

`curl http://localhost:3001/api/parcelles`

*# Récupérer les arbres*

`curl http://localhost:3001/api/arbres`

## Fonctionnalités

### Gestion de la Culture

-  Cartographie des parcelles
-  Suivi des arbres truffiers (espèce, âge, état)
-  Planning des interventions (irrigation, taille, travail du sol)

### Gestion de la Production

-  Enregistrement des récoltes (poids, qualité, localisation)
-  Suivi des ventes
-  Gestion des clients

### Historique & Statistiques

-  Traçabilité complète (triggers automatiques)
-  Statistiques par parcelle, arbre, période
-  Tableaux de bord avec graphiques
-  Export de rapports (à développer)

## Structure de la Base de Données

### Tables principales

- **parcelles** : Informations sur les parcelles
- **arbres** : Inventaire des arbres truffiers
- **interventions** : Planning et historique des travaux
- **recoltes** : Enregistrement des récoltes
- **clients** : Gestion des clients
- **ventes** : Suivi des ventes
- **historique** : Audit trail automatique

## Vues statistiques

- `stats_production_parcelle` : Production par parcelle et année
- `stats_production_arbre` : Production par arbre
- `stats_ventes` : Chiffre d'affaires mensuel



## Commandes utiles

bash

*# Voir les logs*

`docker-compose logs -f`

*# Voir les logs d'un service spécifique*

`docker-compose logs -f backend`

*# Arrêter les services*

`docker-compose down`

*# Arrêter et supprimer les volumes (⚠ efface les données)*

`docker-compose down -v`

*# Redémarrer un service*

`docker-compose restart backend`

*# Accéder à la base de données PostgreSQL*

`docker exec -it truffiere_db psql -U truffiere_user -d truffiere`

*# Sauvegarder la base de données*

`docker exec truffiere_db pg_dump -U truffiere_user truffiere > backup.sql`

*# Restaurer la base de données*

`docker exec -i truffiere_db psql -U truffiere_user truffiere < backup.sql`



## Backend

Le backend utilise `nodemon` en mode développement, donc les changements sont automatiquement détectés.

## Frontend

React Hot Reload est activé, les changements sont visibles immédiatement.

## Ajouter une nouvelle route API

Dans `backend/server.js`, ajoutez :

```
javascript

app.get('/api/ma-route', async (req, res) => {
  try {
    // Votre logique ici
    res.json({ message: 'OK' });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Erreur' });
  }
});
```



## Notes importantes

1. **Données de démonstration** : La base de données contient des données d'exemple (4 arbres, 3 parcelles)
2. **Mots de passe** : Changez les mots de passe dans `docker-compose.yml` pour la production
3. **CORS** : Configuré pour le développement, à ajuster pour la production
4. **PostGIS** : Extension activée pour la gestion des coordonnées GPS (cartographie)



## Sécurité (TODO pour la production)

- ☐ Changer les mots de passe par défaut
- ☐ Ajouter l'authentification JWT
- ☐ Configurer HTTPS
- ☐ Limiter les CORS
- ☐ Ajouter la validation des entrées
- ☐ Implémenter les rôles utilisateurs



## Prochaines étapes

1. Créer les composants React manquants
2. Implémenter la cartographie avec Leaflet
3. Ajouter l'export PDF des rapports
4. Créer un système d'authentification
5. Ajouter des graphiques avancés avec Recharts
6. Implémenter la recherche et les filtres
7. Ajouter la gestion des photos (arbres, truffes)

## Dépannage

### Problème de connexion à la base de données

```
bash

# Vérifier que PostgreSQL est prêt
docker-compose logs postgres
```

### Port déjà utilisé

```
bash

# Changer les ports dans docker-compose.yml
ports:
  - "3002:3001" # Au lieu de 3001:3001
```

### Erreur npm install

```
bash

# Supprimer les node_modules et réinstaller
docker-compose down
rm -rf backend/node_modules frontend/node_modules
docker-compose up --build
```

## Support

Pour toute question, consultez les logs avec `docker-compose logs -f`

---

**Version :** 1.0.0

**Dernière mise à jour :** Décembre 2024